UNIUNEA EUROPEANĂ

GUVERNUL ROMÂNIEI

Instrumente Structurale 2007-2013

Platformă de e-learning și curriculă e-content pentru învățământul superior tehnic

Proiectarea Logică

21. Aplicatii ale numaratoarelor

APLICATII ALE NUMARATOARELOR

IMPLEMENTAREA UNUI AUTOMAT FOLOSIND UN NUMĂRĂTOR CA REGISTRU DE STĂRI

Metoda *numărătorului de salt* presupune implementarea unui automat folosind un numărător ca registru de stări. Această abordare foloseşte componente MSI (*medium scale integrates*), cum ar fi numărătoare sincrone, multiplexoare, decodificatoare. Pentru o mai bună înţelegere, va fi mai întâi prezentată această metoda de implementare ca un concept general, urmând particularizarea pentru procesorul ales.

Numărătoarele sincrone sunt indicate pentru implementarea automatelor, în detrimentul celor asincrone, care pot duce la un comportament anormal al circuitului secvenţial considerat. Cele mai frecvent utilizate numărătoare integrate sunt TTL, de exemplu pentru cazul de faţă – numărătorul binar direct TTL 74163. Pe lângă respectarea unui semnal de tact, numărătoarele sincrone prezintă avantajul implementării unui registru de stări cu mecanisme pentru avansarea stării (count - CNT), resetarea la zero (clear - CLR) şi saltul la o stare nouă (load - LD). Codificarea stării următoare, în loc de implementarea ca o funcţie pe biţi de stare, se poate face asertând în condiţiile necesare semnalele de control CNT, LD şi CLR ale numărătorului.

Tranziţiile de stare susţinute de un numărător sunt prezentate în figura următoare.

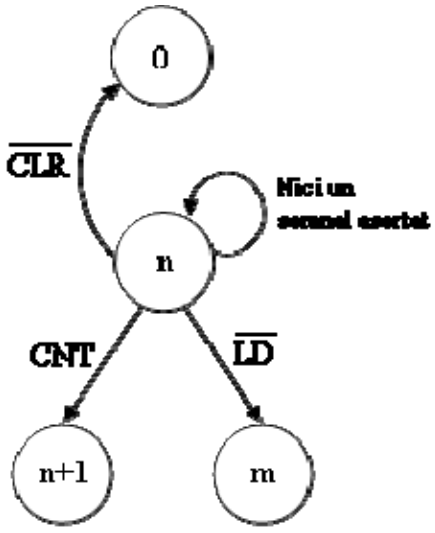


Fig. 12. Tranziţii de stare ale numărătorului

METODA NUMĂRĂTORULUI DE SALT

Numărătoarele de stare se împart în doua clase: *pure* şi *hibride*. Cele pure permit numai una din patru stări următoare posibile: starea curentă (numărătorul aşteaptă), starea secvenţială următoare (numărătorul numără), starea 0 (numărătorul se resetează), sau o stare

singulară de salt (numărătorul încarcă). În acest caz, starea de salt este strict o funcţie de starea curentă. Un numărător hibrid suportă aceleaşi tranziţii, dar permite stării de salt să fie o funcţie atât de intrări, cât şi de starea curentă.

Numărătorul de salt pur

Schema bloc a unui numărător de salt pur este prezentată în figura următoare.

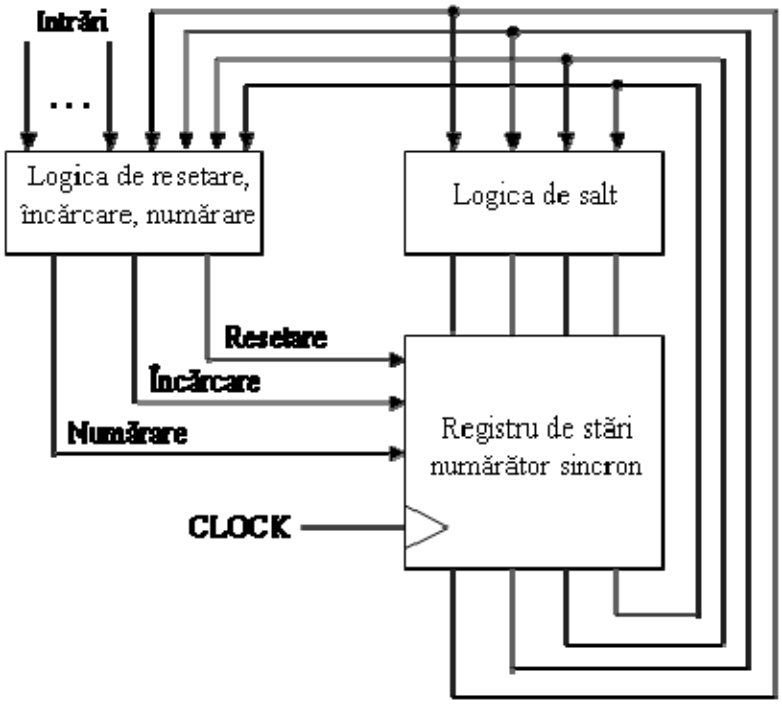


Fig. 13. Schema bloc a unui numărător de salt pur

Starea de salt este o funcţie de starea curentă, în timp ce intrările resetare, încărcare, numărare ale registrului de stări depind de starea curentă şi de intrările curente. Se face presupunerea că resetarea precede încărcarea, care la rândul ei precedă numărarea. Blocurile logice din figură pot fi implementate cu porţi logice discrete, PAL/PLA (*programmable array logic/programmable logic arrays*) sau ROM (*read only memories*). În mod frecvent se folosesc circuite ROM pentru a implementa logica stării următoare.

Secvenţionarea restrânsă a stărilor numărătorului pur este dată în figura 14. Pentru a profita maxim de registrul de stări al numărătorului, trebuie ca stările sa fie desemnate în secvenţă de numărare. Cea mai frecventă trecere a tranziţiei trebuie aleasă 0.

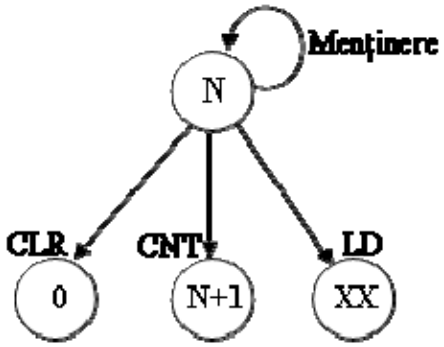
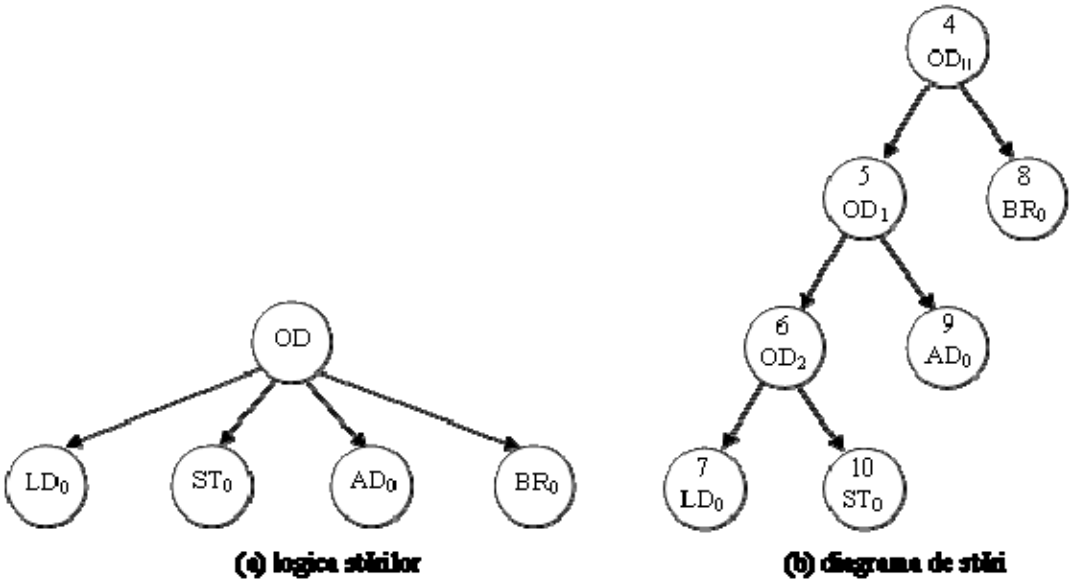


Fig. 14. Secvenţionarea stărilor

Această structură se dovedeşte de obicei a fi resctrivctivă pentru stările care necesită o ramificare multi-căi mai generală. Pentru aceste stări, abordarea prin prisma numărătorului de salt pur va trebui să introducă un surplus de stări, ca în figura următoare.



(a) logica stărilor (b) diagrama de stări

Fig. 15. Efectul ramificării

Figura 15(a) prezintă diagrama de decodificare a operaţiei pentru procesorul considerat. Pentru a implementa această secvenţă folosind un numărător de salt pur, este necesară diagrama de stări din figura 15(b). Astfel se introduc două noi stări de decodificare, şi prin urmare se măreşte numărul de stări necesare executării instrucţiunilor de încărcare, stocare, sau adăugare din memorie.

Din aceste motive a fost conceput modelul numărătorului de salt hibrid, mult mai general decât cel pur.

Numărătorul de salt hibrid

Numărătorul de salt hibrid rezolvă problema ramificării multi-căi prin transformarea stării de salt într-o funcţie atât de intrări, cât şi de starea curentă. Acest lucru este evidenţiat în figura 16. Starea de salt şi logica resetării, a încărcării şi a numărării sunt toate funcţii de intrări şi starea curentă.

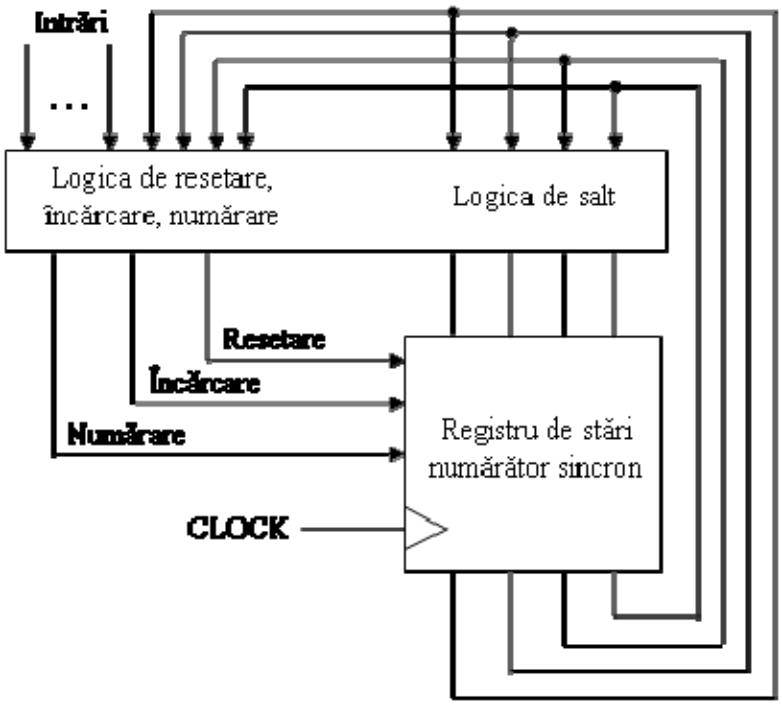


Fig. 16. Schema bloc a unui numărător de salt hibrid

Numărătorul de salt hibrid poate fi implementat folosind o diagramă de stări de tip Mealy. În primul rând trebuie atribuite diagramei stările codate. Cele mai multe tranziţii de stări avansează în starea următoare sau aşteaptă în starea curentă depinzând de valoarea semnalului de aşteptare.

Trebuie de asemena găsită starea căreia îi va fi atribuit 0. Deoarece starea RES este cea mai frecvent utilizată, este cea mai bună posibilitate. Începând cu 0, atribuirea stărilor se va face în sens cu avansarea în diagramă, ca în figura 17.

În continuare se identifică stările de ramificare, ale căror tranziţii către stările următoare nu pot fi descrise prin termeni simpli ca aşteptare (menţinere a stării curente), numărare (avansare în starea următoare), sau resetare (trecere la 0). Singura asemenea stare este OD (decodificarea operaţiei). În acest caz, tranziţiile către stările următoare pot fi descrise ca funcţii ale stării curente şi biţilor de codare ai operaţiilor registrului de instrucţiuni. De fapt, din moment ce aceasta este singura ramură multi-căi din întreaga diagramă de stări, logica stării de salt este o funcţie numai de biţii de codificare ai operaţiilor din registrul de instrucţiuni.

Diagrama completă de stări este prezentată în figura 17. Se presupune că stările sunt numerotate de la S_0 la S_{13}. Pentru tranziţiile către starea RES (reset – S_0), starea CLR a numărătorului trebuie asertată în stările S_7, S_9 (dacă semnalul de *aşteptare* este de asemenea asertat), S_{12} şi S_{13}.

Semnalul LD al numărătorului este asertat numai pentru stări cu ramuri multi-căi. Pentru această diagramă de stări, trebuie asertat în starea S_4, de decodificare a operaţiei (OD). Logica saltului, determinată de biţii de codificare ai operaţiei din IR, generează ca noua stare să fie încărcată în numărătorul de stări.

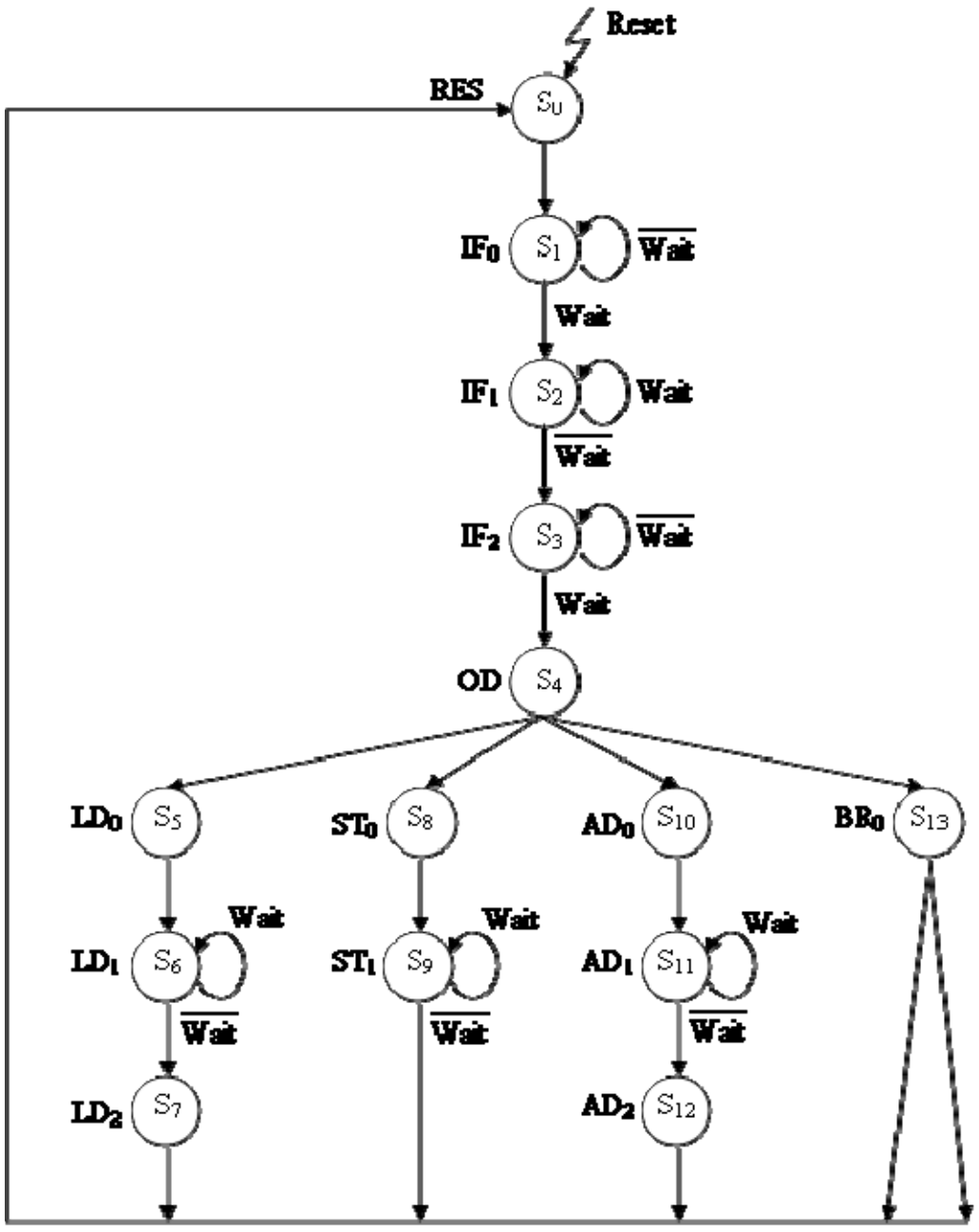


Fig. 17. Atribuirea stărilor numărătorului de salt hibrid

Pentru asertarea semnalului CNT ecuaţiile Boole-ene sunt:

$$\text{CNT} = (S_0 + S_5 + S_8 + S_{10}) + \text{Wait} \cdot (S_1 + S_9) + \text{Wait} \cdot (S_2 + S_6 + S_9 + S_{10})$$

$$\text{HOLD} = \overline{\text{Wait}} \cdot (S_1 + S_9) + \text{Wait} \cdot (S_2 + S_6 + S_9 + S_{11})$$

Trebuie precizată notaţia HOLD, făcută pentru starea de *Menţinere* a numărătorului (figura 14). Starea de menţinere este exact negarea stării de numărare: $\text{HOLD} = \overline{\text{CNT}}$

După cum se observă, ecuaţia pentru HOLD este mai simplă decât cea pentru CNT. Se va implementa aşadar HOLD şi apoi se va face negarea.

Logica stării de salt S_4 (OD) poate fi implementată cu un ROM ale cărui intrări de adrese sunt cei doi biţi de codificare din IR.

Implementarea numărătorului de salt

O descriere schematică a numărătorului de salt este prezentată în figura următoare.

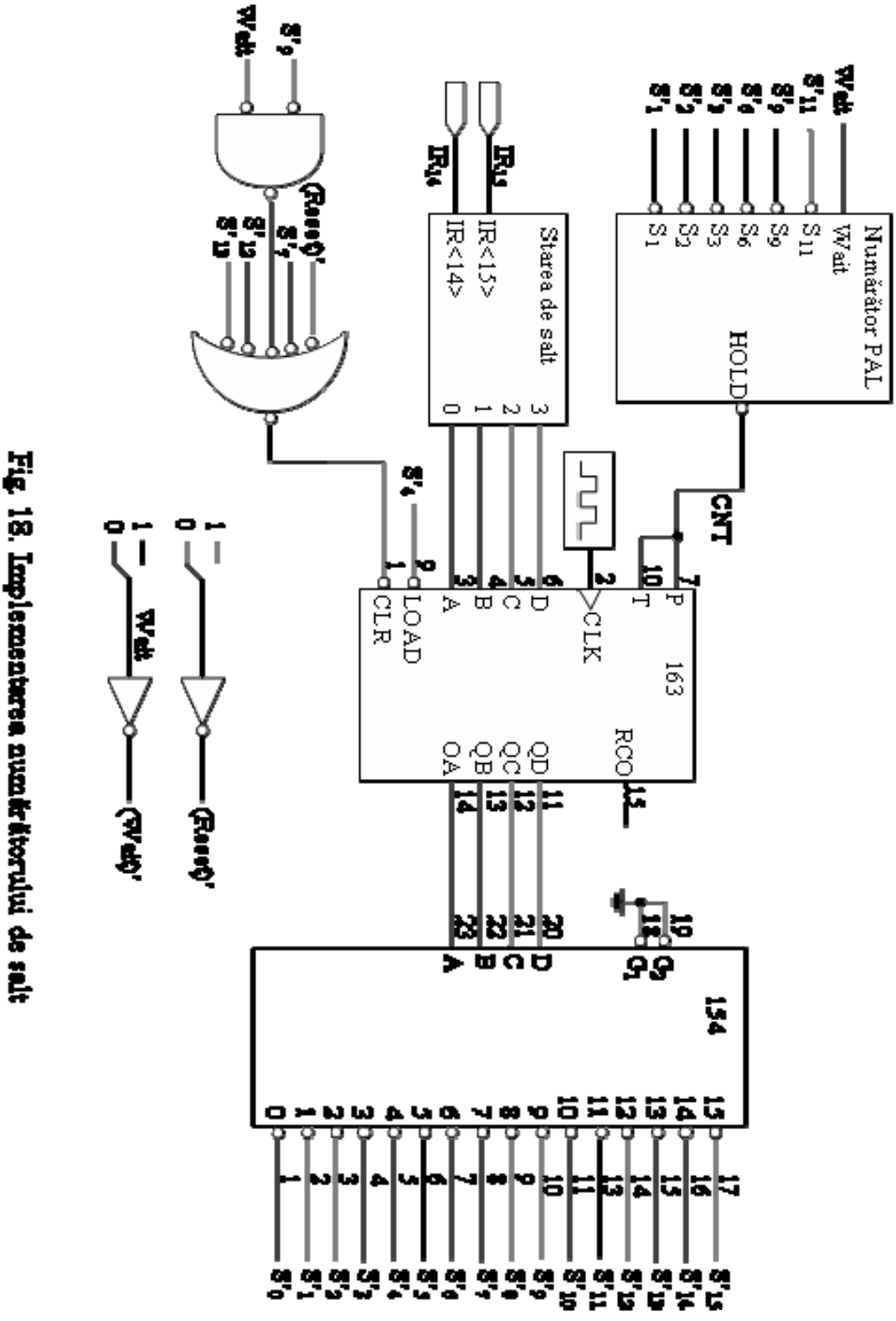


Fig. 18. Implementarea numărătorului de salt

Componentele principale sunt:
(1) un numărător sincron (74163) folosit ca registru de stări
(2) un decodificator cu 4 intrări şi 16 ieşiri (74154) pentru a genera semnalele de indetificare a stării curente
(3) logica stării de salt, implementată cu un circuit ROM indexat de cei doi biţi de codificare a instrucţiunii din IR
(4) un circuit PAL pentru implementarea intrării CNT a numărătorului
(5) logică discretă pentru implementarea intrării CLR a numărătorului

Din moment ce LD este asertat numai într-o singură stare, S_4, ieşirea decodificatorului pentru aceasta stare comandă direct intrarea LD.

Logica numărătorului de salt hibrid poate fi detaliată, mai întâi prin inspectarea conţinutului circuitului ROM. Astfel, pentru fiecare codificare de adresă formată din biţii IR<15:14>, în ROM sunt stocate stările următoare corespunzătoare:

Adresă	Conţinut (stare simbolică)
00	0101 (LD_0)
01	1000 (ST_0)
10	1010 (AD_0)
11	1101 (BR_0)

Realizarea restului numărătorului de salt nu este dificilă. Totuşi, există o complicaţie: polaritatea negativă a semnalelor de control LOAD şi CLR ale numărătorului 74163, precum şi ieşirile active jos ale decodificatorului 74154.

În ceea ce priveşte semnalul CLR, în logică pozitivă acesta poate fi exprimat astfel:

$$\text{CLR} = \text{Reset} + S_7 + S_{12} + S_{19} + (S_{19} \cdot \text{Wait})$$

Deoarece semnalul de intrare CLR este activ jos, funcţia trebuie rescrisă folosind teorema DeMorgan:

$$\overline{\text{CLR}} = \overline{\text{Reset}} \cdot \overline{S_7} \cdot \overline{S_{12}} \cdot \overline{S_{19}} \cdot \overline{(S_{19} + \text{Wait})}$$

Din fericire, decodificatorul furnizează exact aceste semnale active jos. Implementarea se realizează cu o poartă NAND cu 2 intrări negate (ceea ce derivă de fapt o poartă OR) şi o poartă NOR cu 5 intrări negate (ceea ce derivă o poartă AND).

Este mult mai convenabilă implementarea HOLD decât CNT. Cu alte cuvinte, complementul lui HOLD este CNT. Astfel, HOLD poate fi implementat folosind un circuit PAL cu polaritatea ieşirii programabilă şi alegând ca ieşirea să fie în logică negtivă. Funcţia PAL devine:

$$\text{HOLD} = S_1 \cdot \overline{\text{Wait}} + S_2 \cdot \text{Wait} + S_9 \cdot \overline{\text{Wait}} + S_6 \cdot \text{Wait} + S_9 \cdot \text{Wait} + S_{11} \cdot \text{Wait}$$

$$\text{(CNT)}\ \overline{\text{HOLD}} = \overline{S_1 \cdot \overline{\text{Wait}} + S_2 \cdot \text{Wait} + S_9 \cdot \overline{\text{Wait}} + S_6 \cdot \text{Wait} + S_9 \cdot \text{Wait} + S_{11} \cdot \text{Wait}}$$

Deoarece decodificatorul dă starea curentă în logică negativă, este preferată specificarea funcţiei HOLD folosind versiunile active jos ale intrărilor de stare. Circuitul PAL este de fapt implementat astfel:

$$\text{HOLD} = \overline{S_1} \cdot \overline{\text{Wait}} + \overline{S_2} \cdot \text{Wait} + \overline{S_9} \cdot \overline{\text{Wait}} + \overline{S_6} \cdot \text{Wait} + \overline{S_9} \cdot \text{Wait} + \overline{S_{11}} \cdot \text{Wait}$$

$$\text{(CNT)}\ \overline{\text{HOLD}} = \overline{\overline{S_1} \cdot \overline{\text{Wait}} + \overline{S_2} \cdot \text{Wait} + \overline{S_9} \cdot \overline{\text{Wait}} + \overline{S_6} \cdot \text{Wait} + \overline{S_9} \cdot \text{Wait} + \overline{S_{11}} \cdot \text{Wait}}$$

Codificări

Codificarea stărilor şi a semnalelor de ieşire este făcută după cum urmează:

Stare	Cod stare	Ieşire (operaţii de transfer la regiştri)	Simbol ieşire
S_0	0000	0 → PC	C_0
S_1	0001	PC → MAR; PC+1 → PC	C_1
S_2	0010	MAR→Memorie; 1→Citire/(Scriere)'; 1→Cerere	C_2
S_3	0011	Memorie→MBR	C_3
S_4	0100	Memorie→IR	C_4
S_5	0101	IR<13:0>→MAR	C_5
S_6	0110	AC→MBR	C_6
S_7	0111	MBR→AC	C_7
S_8	1000	MAR→Mem.;MBR→Mem.;0→Citire/(Scriere)';1→Cerere	C_8
S_9	1001	0→Cerere	C_9
S_{10}	1010	MBR+AC→AC	C_{10}
S_{11}	1011	IR<13:0>→PC	C_{11}
S_{12}	1100		
S_{13}	1101		

Bistabilul JK

Implementarea unităţii de control s-a realizat şi din punctul de vedere al unui automat clasic, în sensul implementării prin folosirea bistabililor JK.

Circuitele basculante bistabile (bistabilii) sunt circuite cu două stări stabile, trecerea dintr-o stare în alta făcându-se numai la modificarea unei variabile de intrare.

Structura bistabililor JK este prezentată în figura 19. În acest caz, J şi K sunt intrările bistabilului, iar Q şi Q' sunt ieşirile. Tabelul de funcţionare este:

J(t)	K(t)	Q(t)	Q(t+1)
0	0	x	Q(t)
0	1	x	0
1	0	x	1
1	1	x	Q'(t)

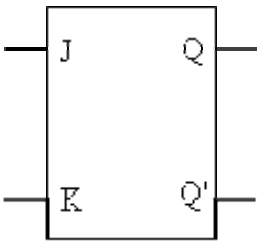


Fig.19. Bistabilul JK

Din tabelul de funcţionare se poate deduce următoarea ecuaţie caracteristică, în care Q(t) reprezintă starea curentă, iar Q(t+1) starea următoare: $Q(t+1) = Q(t)K' + Q'(t)J$

În cele ce urmează au fost realizate tabelele de tranziţie, atât din punct de vedere simbolic, cât şi codificat. În cele din urmă, este prezentată tabela de tranziţie pregătitoare implementării unităţii de control folosind bistabili JK.

Tabela de tranziţie simbolică

Stare	Cod	Simbol	Reset	Wait	IR‹15›	IR‹14›	AC‹15›	Starea curentă	Starea următoare	Operaţii de transfer la regiştri
S_0	0000	RES	1	x	x	x	x	x	RES (S_0)	0 → PC
			0	x	x	x	x	RES (S_0)	IF_0 (S_1)	PC → MAR; PC+1 → PC
S_1	0001	IF_0	0	0	x	x	x	IF_0 (S_1)	IF_0 (S_1)	
			0	1	x	x	x	IF_0 (S_1)	IF_1 (S_2)	MAR→Memorie; 1→Citire/(Scriere)'; 1→Cerere
S_2	0010	IF_1	0	1	x	x	x	IF_1 (S_2)	IF_1 (S_2)	1→Citire/(Scriere)'; 1→Cerere; MAR→Memorie
			0	0	x	x	x	IF_1 (S_2)	IF_2 (S_3)	Memorie→MBR
S_3	0011	IF_2	0	0	x	x	x	IF_2 (S_3)	IF_2 (S_3)	
			0	1	x	x	x	IF_2 (S_3)	OD (S_4)	Memorie→IR
S_4	0100	OD	0	x	0	0	x	OD (S_4)	LD_0 (S_5)	IR<15:14>=00; IR<13:0>→MAR
			0	x	0	1	x	OD (S_4)	ST_0 (S_8)	IR<15:14>=01; IR<13:0>→MAR; AC→MBR
			0	x	1	0	x	OD (S_4)	AD_0 (S_{10})	IR<15:14>=10; IR<13:0>→MAR
			0	x	1	1	x	OD (S_4)	BR_0 (S_{13})	IR<15:14>=11; IR<13:0>→MAR
S_5	0101	LD_0	0	0	x	x	x	LD_0 (S_5)	LD_0 (S_5)	
			0	1	x	x	x	LD_0 (S_5)	LD_1 (S_6)	MAR→Memorie; 1→Citire/(Scriere)'; 1→Cerere
S_6	0110	LD_1	0	1	x	x	x	LD_1 (S_6)	LD_1 (S_6)	1→Citire/(Scriere)'; 1→Cerere; MAR→Memorie
			0	0	x	x	x	LD_1 (S_6)	LD_2 (S_7)	Memorie→MBR
S_7	0111	LD_2	0	0	x	x	x	LD_2 (S_7)	LD_2 (S_7)	
			0	1	x	x	x	LD_2 (S_7)	RES (S_0)	MBR→AC
S_8	1000	ST_0	0	0	x	x	x	ST_0 (S_8)	ST_0 (S_8)	
			0	1	x	x	x	ST_0 (S_8)	ST_1 (S_9)	MAR→Mem.;MBR→Mem.;0→Citire/(Scriere)';1→Cerere
S_9	1001	ST_1	0	1	x	x	x	ST_1 (S_9)	ST_1 (S_9)	0→Citire/(Scriere)';1→Cerere;MAR→Mem.;MBR→Mem.
			0	0	x	x	x	ST_1 (S_9)	RES (S_0)	0→Cerere
S_{10}	1010	AD_0	0	0	x	x	x	AD_0 (S_{10})	AD_0 (S_{10})	
			0	1	x	x	x	AD_0 (S_{10})	AD_1 (S_{11})	MAR→Memorie; 1→Citire/(Scriere)'; 1→Cerere
S_{11}	1011	AD_1	0	1	x	x	x	AD_1 (S_{11})	AD_1 (S_{11})	1→Citire/(Scriere)'; 1→Cerere; MAR→Memorie
			0	0	x	x	x	AD_1 (S_{11})	AD_2 (S_{12})	Memorie→MBR
S_{12}	1100	AD_2	0	0	x	x	x	AD_2 (S_{12})	AD_2 (S_{12})	
			0	1	x	x	x	AD_2 (S_{12})	RES (S_0)	MBR+AC→AC
S_{13}	1101	BR_0	0	x	x	x	1	BR_0 (S_{13})	RES (S_0)	IR<13:0>→PC
			0	x	x	x	0	BR_0 (S_{13})	RES (S_0)	

Tabela de tranziţie

Intrări					Stare curentă				Stare următoare				Ieşiri
Reset	**Wait**	**IR‹15›**	**IR‹14›**	**AC‹15›**	Q^t_3	Q^t_2	Q^t_1	Q^t_0	Q_3^{t+1}	Q_2^{t+1}	Q_1^{t+1}	Q_0^{t+1}	
1	x	x	x	x	x	x	x	x	0	0	0	0	C_0
0	x	x	x	x	0	0	0	0	0	0	0	1	C_1
0	0	x	x	x	0	0	0	1	0	0	0	1	
0	1	x	x	x	0	0	0	1	0	0	1	0	C_2
0	1	x	x	x	0	0	1	0	0	0	1	0	C_2
0	0	x	x	x	0	0	1	0	0	0	1	1	C_3
0	0	x	x	x	0	0	1	1	0	0	1	1	
0	1	x	x	x	0	0	1	1	0	1	0	0	C_4
0	x	0	0	x	0	1	0	0	0	1	0	1	C_5
0	x	0	1	x	0	1	0	0	1	0	0	0	C_5 ; C_6
0	x	1	0	x	0	1	0	0	1	0	1	0	C_5
0	x	1	1	x	0	1	0	0	1	1	0	1	C_5
0	0	x	x	x	0	1	0	1	0	1	0	1	
0	1	x	x	x	0	1	0	1	0	1	1	0	C_2
0	1	x	x	x	0	1	1	0	0	1	1	0	C_2
0	0	x	x	x	0	1	1	0	0	1	1	1	C_3
0	0	x	x	x	0	1	1	1	0	1	1	1	
0	1	x	x	x	0	1	1	1	0	0	0	0	C_7
0	0	x	x	x	1	0	0	0	1	0	0	0	
0	1	x	x	x	1	0	0	0	1	0	0	1	C_8
0	1	x	x	x	1	0	0	1	1	0	0	1	C_8
0	0	x	x	x	1	0	0	1	0	0	0	0	C_9
0	0	x	x	x	1	0	1	0	1	0	1	0	
0	1	x	x	x	1	0	1	0	1	0	1	1	C_2
0	1	x	x	x	1	0	1	1	1	0	1	1	C_2
0	0	x	x	x	1	0	1	1	1	1	0	0	C_3
0	0	x	x	x	1	1	0	0	1	1	0	0	
0	1	x	x	x	1	1	0	0	0	0	0	0	C_{10}
0	x	x	x	1	1	1	0	1	0	0	0	0	C_{11}
0	x	x	x	0	1	1	0	1	0	0	0	0	

Tabela de tranziţie necesară implementării cu bistabili JK

Intrări					Stare curentă				Stare următoare				JK 3		JK 2		JK 1		JK 0	
Reset	**Wait**	**IR‹15›**	**IR‹14›**	**AC‹15›**	Q^t_3	Q^t_2	Q^t_1	Q^t_0	Q_3^{t+1}	Q_2^{t+1}	Q_1^{t+1}	Q_0^{t+1}	J_3	K_3	J_2	K_2	J_1	K_1	J_0	K_0
1	x	x	x	x	x	x	x	x	0	0	0	0	x	x	x	x	x	x	x	x
0	x	x	x	x	0	0	0	0	0	0	0	1	0	x	0	x	0	x	1	x
0	0	x	x	x	0	0	0	1	0	0	0	1	0	x	0	x	0	x	x	0
0	1	x	x	x	0	0	0	1	0	0	1	0	0	x	0	x	1	x	x	1
0	1	x	x	x	0	0	1	0	0	0	1	0	0	x	0	x	x	0	0	x
0	0	x	x	x	0	0	1	0	0	0	1	1	0	x	0	x	x	0	1	x
0	0	x	x	x	0	0	1	1	0	0	1	1	0	x	0	x	x	0	x	0
0	1	x	x	x	0	0	1	1	0	1	0	0	0	x	1	x	x	1	x	1
0	x	0	0	x	0	1	0	0	0	1	0	1	0	x	x	0	0	x	1	x
0	x	0	1	x	0	1	0	0	1	0	0	0	1	x	x	1	0	x	0	x
0	x	1	0	x	0	1	0	0	1	0	1	0	1	x	x	1	1	x	0	x
0	x	1	1	x	0	1	0	0	1	1	0	1	1	x	x	0	0	x	1	x
0	0	x	x	x	0	1	0	1	0	1	0	1	1	x	x	0	0	x	x	0
0	1	x	x	x	0	1	0	1	0	1	1	0	0	x	x	0	1	x	x	1
0	1	x	x	x	0	1	1	0	0	1	1	0	0	x	x	0	x	0	0	x
0	0	x	x	x	0	1	1	0	0	1	1	1	0	x	x	0	x	0	1	x
0	0	x	x	x	0	1	1	1	0	1	1	1	0	x	x	0	x	0	x	0
0	1	x	x	x	0	1	1	1	0	0	0	0	0	x	x	1	x	1	x	1
0	0	x	x	x	1	0	0	0	1	0	0	0	x	0	0	x	0	x	0	x
0	1	x	x	x	1	0	0	0	1	0	0	1	x	0	0	x	0	x	1	x
0	1	x	x	x	1	0	0	1	1	0	0	1	x	0	0	x	0	x	x	0
0	0	x	x	x	1	0	0	1	0	0	0	0	x	1	0	x	0	x	x	1
0	0	x	x	x	1	0	1	0	1	0	1	0	x	0	0	x	x	0	0	x
0	1	x	x	x	1	0	1	0	1	0	1	1	x	0	0	x	x	0	1	x
0	1	x	x	x	1	0	1	1	1	0	1	1	x	0	0	x	x	0	x	0
0	0	x	x	x	1	0	1	1	1	1	0	0	x	0	1	x	x	1	x	1
0	0	x	x	x	1	1	0	0	1	1	0	0	x	0	x	0	0	x	0	x
0	1	x	x	x	1	1	0	0	0	0	0	0	x	1	x	1	0	x	0	x
0	x	x	x	1	1	1	0	1	0	0	0	0	x	1	x	1	0	x	x	1
0	x	x	x	0	1	1	0	1	0	0	0	0	x	1	x	1	0	x	x	1

MINIMIZAREA FUNCŢIILOR LOGICE

Minimizarea funcţiilor logice pentru intrările J şi K ale bistabililor folosiţi au fost făcute utilizând programul de minimizare *Espresso*.

Fişierele de intrare şi de ieşire sunt:

Fişierul de intrare

```
.i 9
.o 8

.ilb  reset  wait  ir15  ir14
ac15 q3 q2 q1 q0
.ob j3 k3 j2 k2 j1 k1 j0 k0

.p 30

1-------- --------
0----0000 0-0-0-1-
00---0001 0-0-0--0
01---0001 0-0-1--1
01---0010 0-0--00-
00---0010 0-0--01-
00---0011 0-0--0-0
01---0011 0-1--1-1
0-00-0100 0--00-1-
0-01-0100 1--10-0-
0-10-0100 1--11-0-
0-11-0100 1--00-1-
00---0101 1--00--0
01---0101 0--01--1
01---0110 0--0-00-
00---0110 0--0-01-
00---0111 0--0-0-0
01---0111 0--1-1-1
00---1000 -00-0-0-
01---1000 -00-0-1-
01---1001 -00-0--0
00---1001 -10-0--1
00---1010 -00--00-
01---1010 -00--01-
01---1011 -00--0-0
00---1011 -01--1-1
00---1100 -0-00-0-
01---1100 -1-10-0-
0---11101 -1-10--1
0---01101 -1-10--1

.e
```

Fişierul de ieşire

```
.i 9
.o 8

.ilb  reset  wait  ir15  ir14
ac15 q3 q2 q1 q0
.ob j3 k3 j2 k2 j1 k1 j0 k0

.p 13

--01-0100 10010000
--11-0100 10000010
--10-0100 10011000
-0---1011 00100101
--00-0-0- 00000010
-0----101 10000000
-1---0-11 00110100
-1---10-- 00000010
-0---1-01 01010001
-1---110- 01010001
-0---0-1- 00000010
-1---0--1 00001001
-----000- 00000010
.e
```

Conform fişierului de ieşire, funcţiile rezultate sunt următoarele:

$$J_3 = (IR15)'(IR14)Q_3'Q_2Q_1'Q_0' + (IR15)(IR14)Q_3'Q_2Q_1'Q_0' + (IR15)(IR14)'Q_3'Q_2Q_1'Q_0' +$$
$$+ (Wait)'Q_2Q_1'Q_0 = \left[(IR14) + (IR15)(IR14)'\right]Q_3'Q_2Q_1'Q_0' + (Wait)'Q_2Q_1'Q_0$$

$$K_3 = (Wait)'Q_3Q_1'Q_0 + WaitQ_3Q_2Q_1'$$

$$J_2 = (Wait)'Q_3Q_2'Q_1Q_0 + WaitQ_3'Q_1Q_0$$

$$K_2 = (IR15)'(IR14)Q_3'Q_2Q_1'Q_0' + (IR15)(IR14)'Q_3'Q_2Q_1'Q_0' + WaitQ_3'Q_1Q_0 +$$
$$+ (Wait)'Q_3Q_1'Q_0 + WaitQ_3Q_2Q_1' =$$
$$= \left[(IR14) \oplus (IR15)\right]Q_3'Q_2Q_1'Q_0' + WaitQ_3'Q_1Q_0 + (Wait)'Q_3Q_1'Q_0 + WaitQ_3Q_2Q_1'$$

$$J_1 = (IR15)(IR14)'Q_3'Q_2Q_1'Q_0' + WaitQ_3'Q_0$$

$$K_1 = (Wait)'Q_3Q_2'Q_1Q_0 + WaitQ_3'Q_1Q_0$$

$$J_0 = (IR15)(IR14)Q_3'Q_2Q_1'Q_0' + (IR15)'(IR14)'Q_3'Q_1' + (Wait)'Q_3'Q_1 + Q_3'Q_2'Q_1' =$$
$$= \left[(IR14) \oplus (IR15)\right]'Q_3'Q_2Q_1'Q_0' + (Wait)'Q_3'Q_1 + (Q_3 + Q_2 + Q_1)'$$

$$K_0 = (Wait)'Q_3Q_2'Q_1Q_0 + WaitQ_3Q_1'Q_0 + WaitQ_3Q_2Q_1' + WaitQ_3'Q_0$$

Se observă identitatea $K_1 = J_2$.

Pentru reducerea numărului de porţi logice necesare implementării acestor funcţii, s-a apelat la simplificări, în sensul utilizării, pe lângă porţile OR şi AND, a porţilor XOR sau NOR.

Pentru minimizarea funcţiilor logice ce descriu ieşirile unităţii de comtrol, i.e. operaţiile de transfer la regiştri, s-a folosit, ca şi în cazul anterior, programul de minimizare *Espresso*. Au fost obţinute următoarele fişiere de intrare şi ieşire:

Fişierul de intrare

```
.i 9
.o 12

.ilb  reset  wait  ir15  ir14
ac15 q3 q2 q1 q0
.ob c0 c1  c2 c3 c4 c5 c6 c7
c8 c9 c10 c11

.p 30

1-------- 100000000000
0----0000 010000000000
00---0001 000000000000
01---0001 001000000000
01---0010 001000000000
00---0010 000100000000
00---0011 000000000000
01---0011 000010000000
0-00-0100 000001000000
0-01-0100 000001100000
0-10-0100 000001000000
0-11-0100 000001000000
00---0101 000000000000
01---0101 001000000000
01---0110 001000000000
00---0110 000100000000
00---0111 000000000000
01---0111 000000010000
00---1000 000000000000
01---1000 000000001000
01---1001 000000001000
00---1001 000000000100
00---1010 000000000000
01---1010 001000000000
01---1011 001000000000
00---1011 000100000000
00---1100 000000000000
01---1100 000000000010
0---11101 000000000001
0---01101 000000000000

.e
```

Fişierul de ieşire

```
.i 9
.o 12

.ilb  reset  wait  ir15  ir14
ac15 q3 q2 q1 q0
.ob c0 c1 c2 c3 c4 c5 c6 c7
c8 c9 c10 c11

.p 15

0-01-0100 000000100000
0---11101 000000000001
00---1011 000100000000
00---1001 000000000100
01---0111 000000010000
01---1100 000000000010
01---0011 000010000000
00---0-10 000100000000
01---101- 001000000000
0----0100 000001000000
01---100- 000000001000
0----0000 010000000000
01---0-10 001000000000
01---0-01 001000000000
1-------- 100000000000

.e
```

Conform fişierului de ieşire, funcţiile rezultate sunt următoarele:

$$C_0 = \text{Reset}$$

$$C_1 = (\text{Reset})' Q_3' Q_2' Q_1' Q_0'$$

$$C_2 = (\text{Reset})' \text{Wait}(Q_3 Q_2' Q_1 + Q_3' Q_1 Q_0' + Q_3' Q_1' Q_0)$$

$$C_3 = (\text{Reset})' (\text{Wait})' (Q_3 Q_2' Q_1 Q_0 + Q_3' Q_1 Q_0)$$

$$C_4 = (\text{Reset})' \text{Wait} Q_3' Q_2' Q_1 Q_0$$

$$C_5 = (\text{Reset})' Q_3' Q_2 Q_1' Q_0'$$

$$C_6 = (\text{Reset})' (\text{IR15})' (\text{IR14}) Q_3' Q_2 Q_1' Q_0'$$

$$C_7 = (\text{Reset})' \text{Wait} Q_3' Q_2 Q_1 Q_0$$

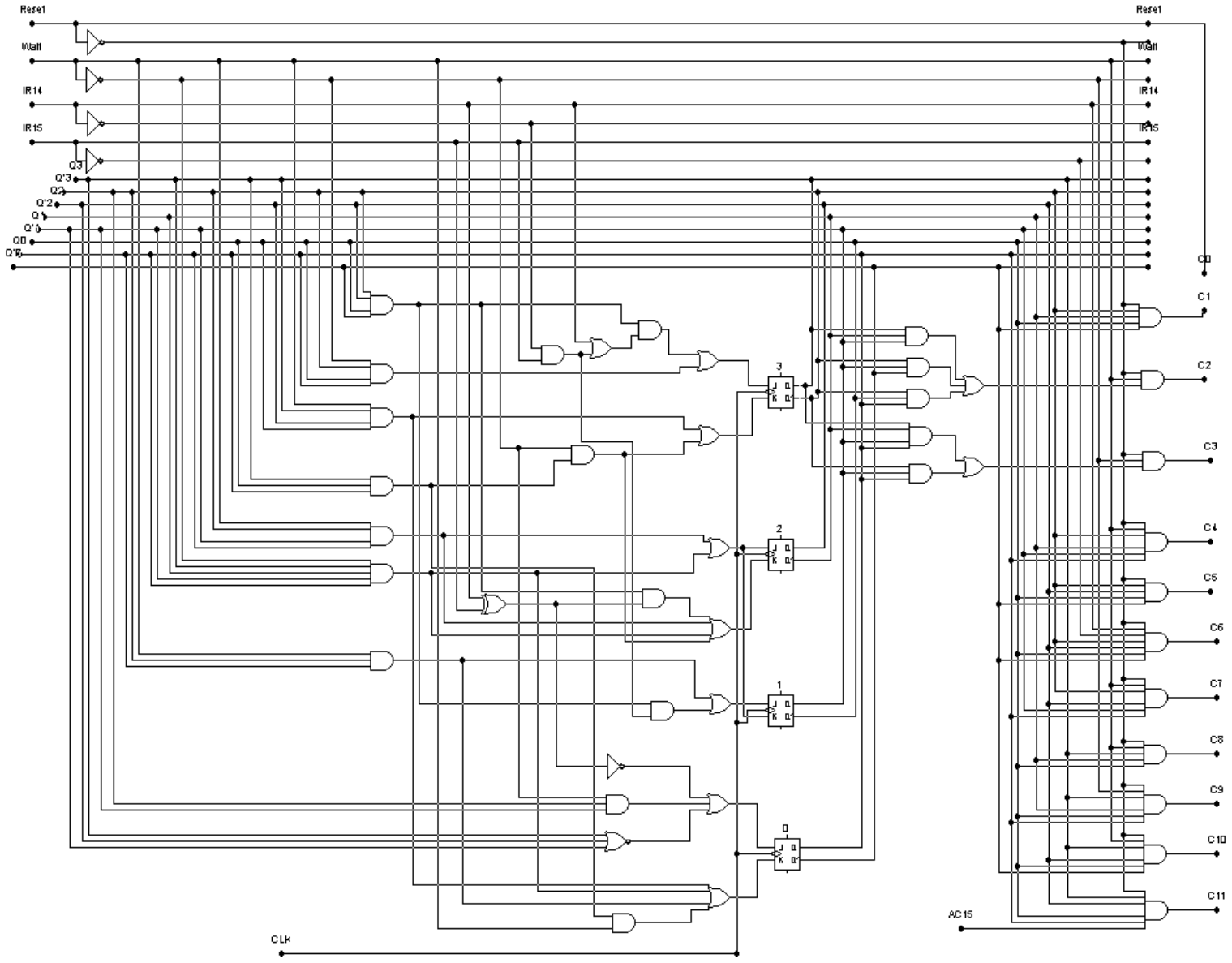
$$C_8 = (\text{Reset})' \text{Wait} Q_3 Q_2' Q_1'$$

$$C_9 = (\text{Reset})' (\text{Wait})' Q_3 Q_2' Q_1' Q_0$$

$$C_{10} = (\text{Reset})' \text{Wait} Q_3 Q_2 Q_1' Q_0'$$

$$C_{11} = (\text{Reset})' (\text{AC15}) Q_3 Q_2 Q_1' Q_0$$

Implementarea unităţii de control



BIBLIOGRAFIE

Ion I. Bucur – *Notiţe de curs*

Randy H. Katz – *Contemporary Logic Design*